DMFQ-A: Dynamic Multilevel Feedback Queue Scheduling with Aging, from CPUs to AI Infrastructure

Final review report · Operating Systems · Aman Golani, Bhargav Mahanta, Kriday Narula

Abstract

Multilevel Feedback Queue (MLFQ) scheduling keeps short jobs responsive without knowing job lengths in advance. Its fixed rules, though, let long jobs sink and wait. This project studies DMFQ-A, an MLFQ variant with two additions: load-adaptive time quanta and selective, level-by-level aging. We evaluate it in a controlled four-policy ablation on a C11 discrete-event CPU scheduler.

We then carry the same rules into two places where the unknown-length problem matters today: large language model (LLM) inference serving and GPU training clusters. A seeded Python prototype compares FCFS, MLFQ, Dynamic MLFQ, Aging MLFQ and DMFQ-A on synthetic workloads at 60–100% load.

Results at about 90% load:

- **Head-of-line blocking:** every feedback-queue policy removes it. The p99 time to first token for short chats drops from 7.85 s to 0.07 s, about 110× lower.
- **Swaps vs. stalls (LLM serving):** DMFQ-A makes 29% fewer KV-cache swaps than MLFQ. Its longest stream stall is 2.9 s, against 4.3 s without aging.
- **GPU cluster:** aging cuts the worst wait for large training jobs from 24.3 h under FCFS to 2.7 h. But DMFQ-A makes 63% more preemptions than MLFQ there.
- **Near saturation:** fixed aging thresholds cause swap thrashing.

The interaction between quantum resizing and aging therefore depends on the workload. The next step is load-aware aging thresholds.

1 Introduction

A CPU scheduler juggles three goals that cannot all be maximised at once: fast first response, fair service for long jobs, and a manageable number of context switches. MLFQ, introduced with time-sharing systems [2] and described in detail in OSTEP [1], resolves this without knowing job lengths. New work starts at the highest priority, and work that uses its full time slice is demoted. Short and interactive jobs therefore finish quickly.

The same rules carry a known risk. A job that keeps using its full quantum moves to the bottom queue. If short jobs keep arriving, it can wait for a very long time. Periodic priority boosts and aging reduce this starvation, but they add context switches and can slow first response.

This problem is no longer confined to operating-system kernels. Two kinds of AI infrastructure have the same shape:

- **LLM inference servers.** They share a GPU among many requests whose output lengths are unknown until generation ends [8], [9].
- **GPU training clusters.** They share accelerators among jobs whose run times are unknown until they converge [7], [13].

Research systems already use multilevel feedback queues in both places [6], [7].

This report makes three contributions:

1. A controlled ablation of four MLFQ policies on a deterministic CPU scheduler, separating the effect of adaptive quanta from the effect of aging.
2. A mapping of DMFQ-A onto LLM serving and GPU clusters, with a reproducible simulator for both.
3. Evidence that the quantum-resizing × aging interaction changes sign between workloads, which motivates load-aware aging.

2 Literature review

Combining dynamic quanta with aging has been done before, both for CPUs and for AI systems. Novelty therefore cannot rest on the combination alone.

CPU scheduling

Source	Contribution	Open challenge
OSTEP, ch. 8 [1]	MLFQ rules and periodic priority boost	Tuning remains workload-sensitive
Behera et al., 2012 [3]	Dynamic time quantum	Reported gains may not generalise across workloads
Dynamic MLFQ with Aging Criteria, 2020 preprint [4]	Dynamic policy combined with aging	Existing integrated prior art
Meing and Tandingan, 2025 [5]	Regression-based quantum adjustment across MLFQ combinations	Different data, model and assumptions

Schedulers in AI systems

System	Scheduling idea	Open question for DMFQ-A
FastServe, NSDI '26 [6]	Skip-join MLFQ preempting at each output token; starved jobs promoted after a fixed threshold α (default 300 ms)	Quanta double per level and α is fixed; neither reacts to load
Tiresias, NSDI '19 [7]	Discretised two-dimensional LAS: two queues by attained GPU-time; PromoteKnob rescues jobs that wait too long	One hand-set knob trades mean JCT for fairness
Orca, OSDI '22 [8]; vLLM, SOSP '23 [9]	Iteration-level batching; paged KV-cache memory; requests served first come, first served	FCFS order causes head-of-line blocking at high load
VTC, OSDI '24 [10]	Fair share across clients, counted in tokens served	Fair across tenants, not across request lengths
Learning to Rank, NeurIPS '24 [11]	Predicts relative output length to approximate shortest-job-first	Needs a trained predictor per model and workload
PIAS, NSDI '15 [12]	MLFQ in commodity datacenter switches; flows demoted by bytes sent	Same unknown-size problem, in networking

FastServe and Tiresias show that MLFQ with a fixed starvation knob already runs in AI-systems research. What remains untested is whether quanta that respond to queue pressure change the trade-off between stalls and swaps.

3 Problem statement, research gap and objectives

Traditional MLFQ fixes its queue quanta and boost cadence in advance. That causes three problems:

1. Fixed quanta ignore changing ready-queue pressure.
2. Low-priority jobs may wait for long periods.
3. Frequent boosts reduce starvation but add switches and can hurt first response.

We do not assume any one scheduler wins. The study measures the trade-offs.

Gap. Earlier work presents dynamic quanta plus aging as a new algorithm. The open contribution is evaluative: separating what each mechanism does, and testing whether the result transfers to modern AI infrastructure.

Objectives

1. Run a four-policy controlled ablation: MLFQ, Dynamic MLFQ, Aging MLFQ and DMFQ-A.
2. Measure fairness and response deterministically: average waiting, turnaround, first response, maximum continuous ready wait and context switches.
3. Test the interaction between quantum resizing and aging.
4. Make every experiment reproducible from a seed.
5. Transfer the policies to LLM inference serving and GPU training clusters.
6. Build an interactive multi-page demo so an operator can see the trade-off before choosing a policy.

4 Proposed algorithm: DMFQ-A

DMFQ-A keeps the three-level MLFQ structure and changes two things: how long a quantum lasts at the lower levels, and how a waiting job climbs back up.

Base rules (shared by all four policies)

1. A new job enters Q0.
2. The CPU always serves the highest non-empty queue; within a queue, jobs run round-robin.
3. A job that uses its whole quantum is demoted one level.
4. A higher-priority arrival preempts the running job. The preempted job keeps its remaining quantum.

Adaptive quanta. Q0 uses a fixed 4 ms quantum. Q1 and Q2 start from base quanta b = 8 ms and 16 ms. These grow with R, the number of other jobs in the READY queue, up to double the base:

```
q = \min\left(2b,\; b + \left\lfloor \frac{b \cdot \min(R, \mathit{ref})}
{\mathit{ref}} \right\rfloor\right), \qquad \mathit{ref} = 8
```

Under heavy load, the CPU switches less often. Under light load, the quanta stay short and responsive.

Selective aging. A job that has waited continuously in the READY queue past its level's threshold moves up one level: Q1 → Q0 after 24 ms, Q2 → Q1 after 48 ms. Promotion is one level at a time rather than a global boost, so only jobs that actually waited are lifted.

Ablation. Each mechanism can be switched off independently:

- MLFQ: neither mechanism
- Dynamic MLFQ: adaptive quanta only
- Aging MLFQ: aging only
- DMFQ-A: both

For the AI case studies we add FCFS as the industry baseline. All policies run on identical seeded workloads.

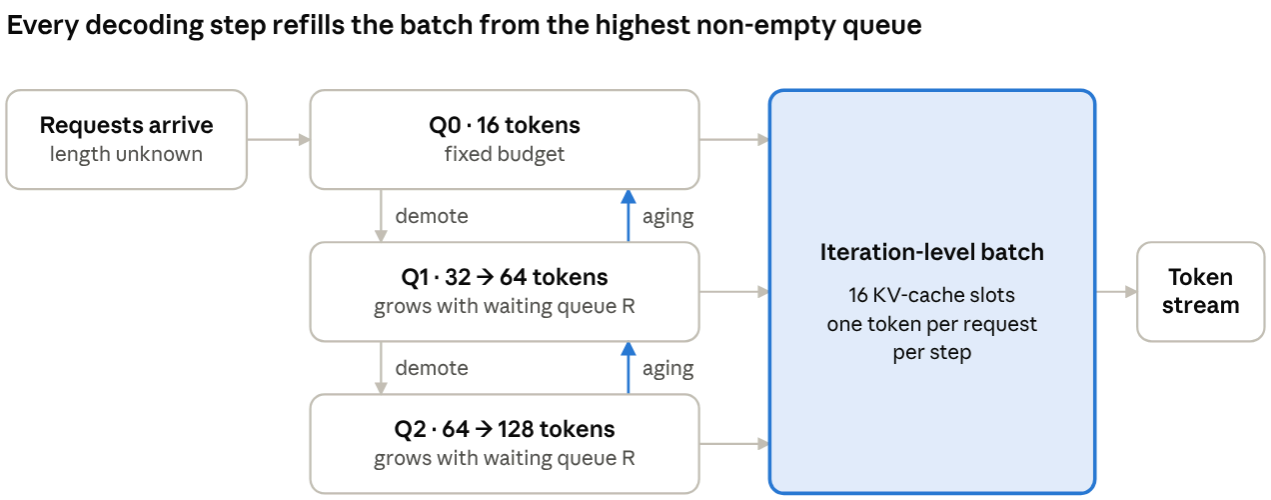
5 Application to AI infrastructure

The same rules apply wherever a shared resource serves jobs of unknown length. Only the unit of the quantum and the cost of a switch change.

Concept	CPU (core study)	LLM inference serving	GPU training cluster
Unit of work	Process	Request: prompt, then output tokens	Training job on 1–16 GPUs
Unknown in advance	CPU burst length	Number of tokens it will generate	Time to converge
Quantum	Time slice (ms)	Token budget per level	Attained GPU-minutes per level
Context switch	Save registers and cache state	Swap the KV cache to host memory	Checkpoint, then restore
Starvation looks like	Process never dispatched	Long answer freezes mid-stream	Large job queued for hours
First response	First dispatch	Time to first token (TTFT)	Queueing delay to start

5.1 LLM inference serving

Modern engines schedule at iteration level [8]: after every decoding step, the scheduler may change which requests sit in the batch. DMFQ-A plugs into that decision.



Aging lifts a waiting request one level: Q1 to Q0 after 1.5 s, Q2 to Q1 after 3 s

DMFQ-A in an iteration-level LLM serving engine · 3 queues, 1 batch

A new request enters Q0 with a 16-token budget, and each decoding step spends one token. A request that spends its budget drops a level, so long generations sink below new chats. A higher-priority request evicts a lower one from the batch, and that eviction is a KV-cache swap whose cost grows with context length. Adaptive quanta enlarge the Q1 and Q2 budgets when many requests are waiting, which reduces evictions. Aging lifts a request whose stream has stalled past its threshold.

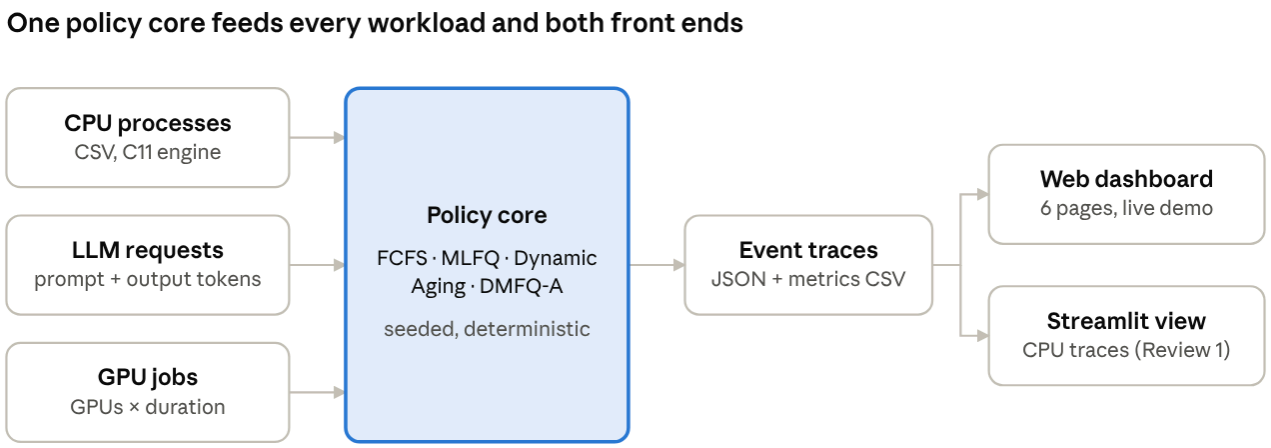
5.2 GPU training clusters

Tiresias [7] already orders jobs by attained GPU service, and DMFQ-A maps onto it directly:

- **Quantum:** attained GPU-minutes. Q0 gets 60; Q1 and Q2 have bases of 240 and 960 and grow to 480 and 1,920 under load. A 16-GPU job uses up Q0 in under 4 minutes; a 1-GPU job gets an hour.
- **Preemption:** a checkpoint, plus 2 minutes to restore when the job resumes.
- **Aging:** Q1 → Q0 after 1 h of waiting, Q2 → Q1 after 2 h. This resembles Tiresias' PromoteKnob, but promotes one level at a time.
- **Placement:** gang (all-or-nothing) on 64 GPUs. Higher levels are placed first, and smaller jobs backfill the gaps.

6 System architecture, implementation and demo

One set of policy rules drives three workload types. Every run writes the same trace format, which both front ends read.



DMFQ-A system architecture · 3 workload adapters, 2 front ends

CPU core study. The Review 1 implementation reads a CSV process workload into a C11 discrete-event scheduler with four policy modules. It writes a JSON trace and per-process metrics. A Python bridge analyses them, and Streamlit visualises them. Model assumptions: a single CPU, no I/O, context switches counted, zero switch time.

AI case studies. `dmfqa_ai_sim.py` is a seeded Python prototype of the same rules. The LLM model advances one decoding iteration at a time: each request in the batch gains one token per step. The step costs 12 ms plus 0.6 ms per sequence, plus prefill and KV-swap costs. The GPU model advances in one-minute steps with gang placement and checkpoint costs. Quanta, the adaptive-quantum formula and aging thresholds follow Section 4, rescaled to tokens or GPU-minutes.

Demo website. A multi-page web dashboard replaces the single Streamlit page for the final demo. The policy core is ported to TypeScript and runs in the browser, so the site deploys as a static site with no backend.

Page	What it shows
Overview	The problem, the idea in one diagram, and headline results
Scheduler Lab	CPU workload editor and Gantt charts for the four policies, side by side
LLM Serving Live	Chat and long requests stream through Q0–Q2 and 16 KV slots, with live TTFT, stall and swap counters
GPU Cluster	A 64-GPU grid filling with jobs by level; JCT and longest-wait meters
Results Explorer	The load sweeps and trade-off charts from Section 8, filterable by policy and load

Page	What it shows
Method & References	Rules, parameters, assumptions and citations

7 Experimental setup

All AI case-study runs use five seeds (11, 23, 37, 41, 53) per load point, and we report the mean. Within a seed, every policy sees the identical workload. The workloads are synthetic distributions chosen to resemble chat and training mixes; they are not replays of production traces.

Parameter	LLM serving testbed	GPU cluster testbed
Jobs per run	1,500 requests	500 training jobs
Arrivals	Poisson, 1.4–2.2 req/s (about 60% load to saturation)	Poisson, 5–7 jobs/h (about 72–81% GPU utilisation)
Job mix	80% chat: output lognormal, median 120 tokens; 20% long: median 900 tokens; prompts median 250 / 1,200 tokens	1 / 2 / 4 / 8 / 16 GPUs at 35 / 20 / 20 / 15 / 10%; duration lognormal, median 60 min, capped at 3 days
Capacity	16 KV-cache slots; step = 12 ms + 0.6 ms per sequence	64 GPUs, gang placement, 1-minute steps
Switch cost	Prefill 0.02 ms per prompt token; KV swap-in 0.004 ms per cached token	2-minute restore after a preemption
Quanta (Q0 / Q1 base / Q2 base)	16 / 32 / 64 tokens	60 / 240 / 960 GPU-minutes
Aging (Q1 → Q0 / Q2 → Q1)	1.5 s / 3 s	1 h / 2 h
Long-job cut-off	More than 512 output tokens	More than 960 GPU-minutes

Metrics. Each CPU metric has a direct counterpart:

- **First response:** time to first token (TTFT) for requests; queueing delay for training jobs.
- **Turnaround:** end-to-end latency for requests; job completion time (JCT) for training jobs.
- **Maximum continuous READY wait:** the longest stall of a long request, or the longest wait of a long job.

- **Context switches:** preemptions (KV-cache swaps or checkpoints).
- **Throughput:** tokens per second, and mean latency per output token.

8 Results and discussion

This section covers the AI case studies. [Insert the CPU ablation results from the C11 runs here: average waiting, turnaround, first response, maximum continuous READY wait and context switches for the four policies.]

8.1 LLM serving at about 90% load

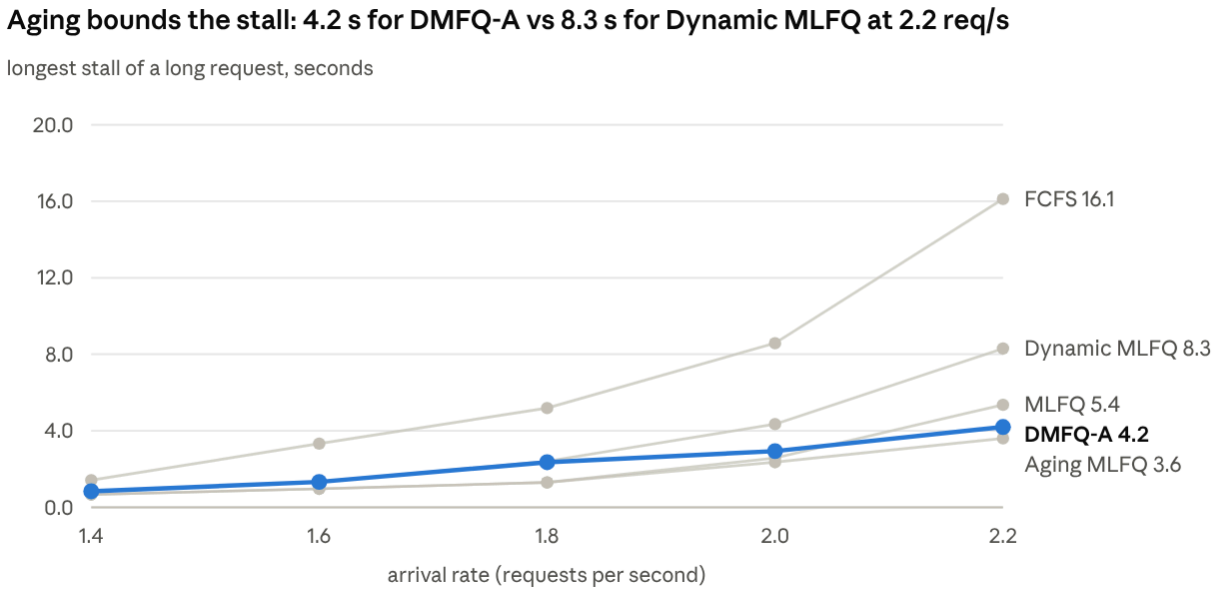
Any feedback-queue policy removes head-of-line blocking. The swap-versus-stall trade-off is where the policies differ.

Policy (2.0 req/s)	Short-chat p99 TTFT	Long-request p99 latency	Longest stall, long request	Mean latency per token	Preemptions (KV swaps)
FCFS	7.85 s	41.8 s	8.6 s	29.0 ms	0
MLFQ	0.07 s	61.9 s	2.6 s	25.8 ms	3,247
Dynamic MLFQ	0.07 s	58.2 s	4.3 s	24.3 ms	2,168
Aging MLFQ	0.07 s	62.1 s	2.4 s	25.8 ms	3,255
DMFQ-A	**0.07 s**	**58.8 s**	**2.9 s**	**24.3 ms**	**2,290**

Throughput was about 601 tokens/s for every policy, so the differences above are pure ordering effects.

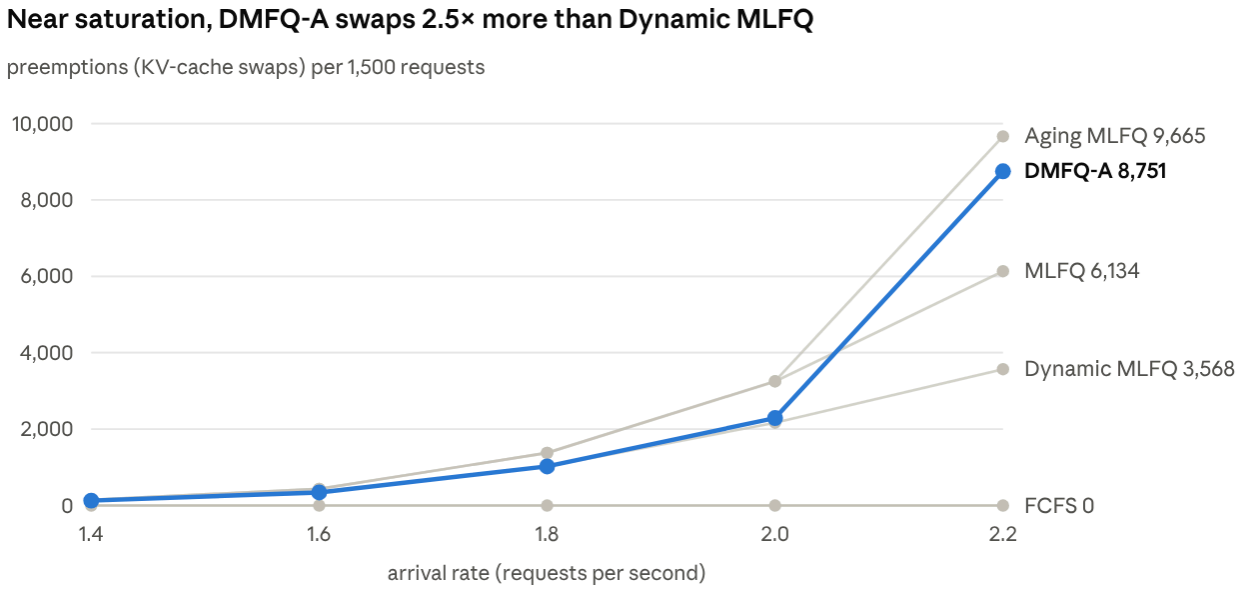
- **Head-of-line blocking.** Under FCFS, a short chat waits behind long generations. Every MLFQ variant serves it within 0.07 s at p99, about 110× faster.
- **The cost.** Long requests finish later, because they now yield to newer chats: p99 latency rises from 41.8 s under FCFS to 58–62 s.
- **Adaptive quanta.** Dynamic MLFQ makes 33% fewer swaps than MLFQ, but its longest stall grows to 4.3 s.
- **Aging.** Aging gives the shortest stall (2.4 s) but the most swaps.
- **DMFQ-A.** It sits between the two: 29% fewer swaps than MLFQ, a 2.9 s longest stall, and the joint-lowest mean latency per token.

8.2 LLM serving under rising load



dmfqa_ai_sim.py · LLM sweep, 1.4–2.2 req/s, mean of 5 seeds

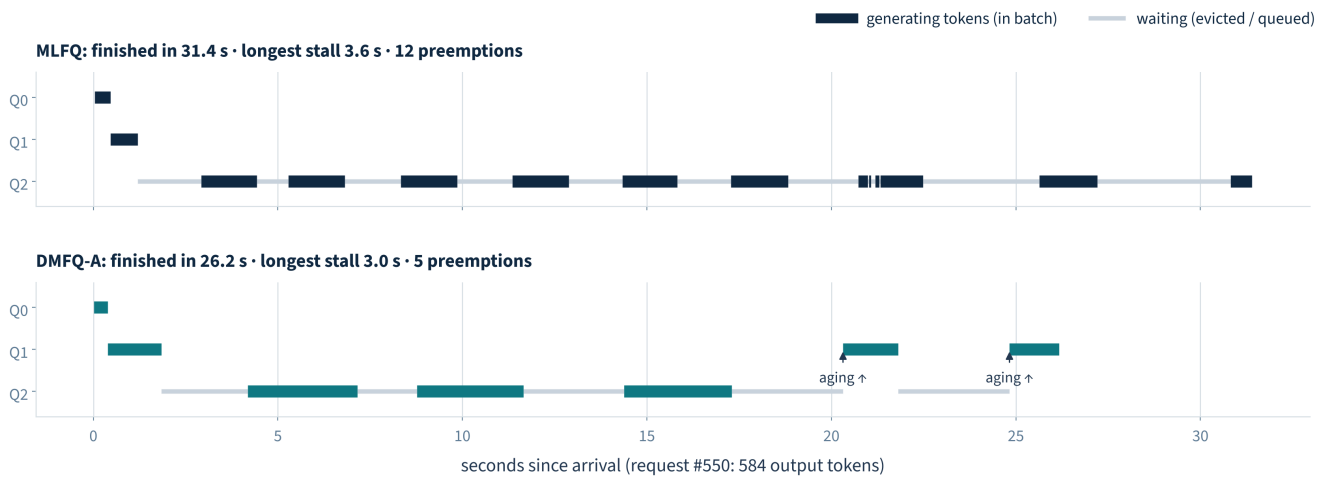
At low load (1.4–1.6 req/s) no request waits long enough to trigger aging, so each aging policy behaves exactly like its non-aging twin. Aging starts to matter around 1.8 req/s. At 2.2 req/s it halves the longest stall relative to Dynamic MLFQ.



dmfqa_ai_sim.py · LLM sweep, 1.4–2.2 req/s, mean of 5 seeds

The same thresholds become harmful near saturation. Promoted requests evict others, which are then promoted in turn, so swaps climb steeply. A threshold tuned for normal load (1.5 s / 3 s) is too short at peak. This is the clearest argument for scaling the aging threshold with load.

8.3 One request, two schedulers



Request #550 (584 output tokens) in the 2.0 req/s workload. Under MLFQ, it is evicted 12 times and finishes in 31.4 s. Under DMFQ-A, larger Q2 budgets mean fewer, longer turns, and aging lifts it back to Q1 twice: 5 preemptions, finished in 26.2 s.

8.4 GPU training cluster at about 77% utilisation

In the cluster, aging bounds the wait for large jobs, but here DMFQ-A preempts more often than MLFQ.

Policy (6 jobs/h)	Short-job mean JCT	Long-job p95 JCT	Long-job longest wait	Preemptions
FCFS	10.2 h	43.0 h	24.3 h	0
MLFQ	1.9 h	51.6 h	5.7 h	786
Dynamic MLFQ	1.8 h	50.5 h	9.0 h	635
Aging MLFQ	1.8 h	53.0 h	2.3 h	1,093
DMFQ-A	**1.9 h**	**54.3 h**	**2.7 h**	**1,282**

- Feedback queues cut mean JCT for short jobs 5.5×, because strict FIFO lets one large gang job block everyone behind it.
- Aging brings the worst wait for large jobs from 24.3 h (FCFS) and 5.7 h (MLFQ) down to 2.3–2.7 h.
- Unlike LLM serving, adding adaptive quanta to aging *increases* preemptions here: 1,282 versus 1,093 for Aging MLFQ. A likely cause: larger budgets let promoted jobs hold GPUs longer, so more running jobs are displaced.

8.5 What the results mean

1. The large, reliable win comes from preemptive feedback queues themselves, in both domains.
2. Aging is what bounds starvation; adaptive quanta alone make it worse.
3. The quantum-resizing × aging interaction changes sign between workloads: fewer swaps for token-level LLM serving, more preemptions for coarse-grained GPU jobs. Objective 3 is answered: the interaction is real and workload-dependent.
4. Fixed aging thresholds break near saturation. The next design step is load-aware aging, not a further combination of mechanisms.

9 Conclusion, limitations and future work

DMFQ-A is best pitched as a tunable scheduling layer for AI infrastructure, not as a new CPU algorithm. Inference platforms and shared GPU clusters both need short work to stay fast while long work still finishes, and both lack advance knowledge of job length.

The evidence supports three uses:

- **LLM inference engines:** replace the FCFS waiting queue with DMFQ-A levels at each decoding step.
- **GPU cluster managers:** use DMFQ-A as the job-queue policy.
- **Campus and lab clusters:** a few GPUs shared by many student jobs, which is exactly where one large run can block everyone else.

The deliverable is a policy library, a reproducible simulator, and an operator dashboard for choosing the trade-off before deploying.

Limitations

- Workloads are synthetic distributions, not production traces.
- The cost model for decoding, prefill and swaps is linear.
- The LLM study models one serving node, without prefix caching or chunked prefill.
- GPU placement ignores network topology.
- The AI case studies use a Python prototype rather than the C11 engine.

Future work

1. **Load-aware aging.** Scale the aging threshold with READY pressure, as the quanta already are, to remove thrashing near saturation.
2. **SLO-aware thresholds.** Derive aging limits from a time-to-first-token target, following FastServe's SLO-tuned α [6].
3. **Real traces.** Replay public chat-length data and the Philly GPU-cluster trace [13].

4. **Engine prototype.** Implement the policy inside an open-source serving engine and measure real swap costs.

References

1. R. H. Arpaci-Dusseau and A. C. Arpaci-Dusseau, *Operating Systems: Three Easy Pieces*, ch. 8, "Scheduling: The Multi-Level Feedback Queue." Arpaci-Dusseau Books, 2018.
2. F. J. Corbató, M. Merwin-Daggett and R. C. Daley, "An experimental time-sharing system," in *Proc. AFIPS Spring Joint Computer Conf.*, 1962.
3. Behera et al., dynamic time quantum for multilevel feedback queues, 2012. [Complete from the Review 1 citation.]
4. "Developed Dynamic Multilevel Feedback Queue with Aging Criteria," preprint, 2020. [Complete from the Review 1 citation.]
5. L. W. C. Meing and D. R. Tandingan Jr., "Exploring multilevel feedback queue combinations and regression-based time quanta in scheduling algorithms," *J. Inf. Syst. Eng. Manag.*, vol. 10, no. 6s, 2025, doi: 10.52783/jisem.v10i6s.699.
6. B. Wu, Y. Zhong, Z. Zhang, S. Liu, F. Liu, Y. Sun, G. Huang, X. Liu and X. Jin, "FastServe: Iteration-level preemptive scheduling for large language model inference," in *Proc. USENIX NSDI*, 2026, pp. 57–74. Preprint: arXiv:2305.05920.
7. J. Gu, M. Chowdhury, K. G. Shin, Y. Zhu, M. Jeon, J. Qian, H. Liu and C. Guo, "Tiresias: A GPU cluster manager for distributed deep learning," in *Proc. USENIX NSDI*, 2019, pp. 485–500.
8. G.-I. Yu, J. S. Jeong, G.-W. Kim, S. Kim and B.-G. Chun, "Orca: A distributed serving system for transformer-based generative models," in *Proc. USENIX OSDI*, 2022.
9. W. Kwon, Z. Li, S. Zhuang, Y. Sheng, L. Zheng, C. H. Yu, J. E. Gonzalez, H. Zhang and I. Stoica, "Efficient memory management for large language model serving with PagedAttention," in *Proc. ACM SOSP*, 2023.
10. Y. Sheng, S. Cao, D. Li, B. Zhu, Z. Li, D. Zhuo, J. E. Gonzalez and I. Stoica, "Fairness in serving large language models," in *Proc. USENIX OSDI*, 2024.
11. Y. Fu, S. Zhu, R. Su, A. Qiao, I. Stoica and H. Zhang, "Efficient LLM scheduling by learning to rank," in *Proc. NeurIPS*, 2024, pp. 59006–59029.
12. W. Bai, L. Chen, K. Chen, D. Han, C. Tian and H. Wang, "Information-agnostic flow scheduling for commodity data centers," in *Proc. USENIX NSDI*, 2015.
13. M. Jeon, S. Venkataraman, A. Phanishayee, J. Qian, W. Xiao and F. Yang, "Analysis of large-scale multi-tenant GPU clusters for DNN training workloads," in *Proc. USENIX ATC*, 2019.